

BRD

BUSINESS REQUIREMENTS DOCUMENT

BRD Agent Suite

Field	Value
Document ID	BRD-BRD-001
Version	1.0
Date	[Date]
Author	[Author]
Status	Draft — pending stakeholder review
Scope of this document	End-to-end requirements and system capabilities for Phase 0 through Phase 3 of the BRD Agent Suite
Target platforms	Local Windows host environment, modern web browsers
Reference	[Reference/supersedes — none for v1.0]

Top Five Features

#	Feature	The question it answers	Section
1	Single-Source Multi-Artifact Generation	"How do we generate a full suite of business, technical, visual, and executive project documentation without manually duplicating data across multiple disconnected files?"	4.1
2	Automated Cross-Documents Consistency Validation	"How do we guarantee that technical specifications, architecture diagrams, and business requirements never contradict each other across generated artifacts?"	4.2
3	Per-Field AI Auto-Generation & Enrichment	"Can the system intelligently draft, infer, or expand specific requirement fields when authoring inputs are only partially provided?"	4.1
4	Native DOCX Export with Embedded Diagrams	"Can stakeholders export fully styled, distribution-ready DOCX and PDF deliverables containing rendered visual flowcharts and architecture diagrams?"	4.2

#	Feature	The question it answers	Section
5	Non-Destructive Iterative Revision	"How do we request natural language modifications on previously generated outputs without losing historical versions and run context?"	4.2

1. Purpose

Product managers, business analysts, and engineering leads must author a Business Requirements Document (BRD), Technical Specifications Document (TSD), visual architecture/process flow diagrams, and an executive one-pager for every new project initiative. Currently, these assets are written manually across disjointed tools such as Microsoft Word and Confluence. Because there is no unified structured source of truth, business documents and technical specifications frequently diverge on project scope, technology selections, and delivery timelines over the lifecycle of a project.

This misalignment results in recurring friction during stakeholder review cycles, delayed architecture review board approvals, and costly downstream engineering rework when implementation teams build against outdated or conflicting specifications. Authoring these four document types by hand is also slow and inconsistent across authors, creating an organizational bottleneck that delays project kickoff.

This document defines the functional and non-functional requirements for the BRD Agent Suite. The software provides a local, automated documentation platform that ingests project inputs once and generates a complete, mutually consistent deliverable package.

2. Objectives

ID	Objective	Success measure
OBJ-01	Cut BRD/TSD authoring time from days to minutes	A complete BRD+TSD+diagrams+one-pager set generated in under 5 minutes from a filled form
OBJ-02	Eliminate drift between the business and technical documents	Consistency check reports zero contradictions across generated document sets

9/16/26, 2:16 AMBRD Agent Suite

ID	Objective	Success measure
OBJ-03	Let non-technical users produce a credible first-draft BRD	A user with no prior BRD-writing experience produces a document matching professional structure on the first attempt
OBJ-04	Standardize requirements architecture across product and engineering teams	100% of generated requirement specifications adhere strictly to unified traceable schemas and export formats

3. Scope

3.1 In scope

- Structured web form and conversational free-text chat interfaces for inputting project parameters.
- Automated multi-agent generation of five synchronized deliverables: BRD, TSD, Mermaid flowchart, Mermaid architecture diagram, and executive one-pager.
- Per-field controls for AI auto-generation/enrichment and explicit "not applicable" section omission.
- Support for custom user-authored additional sections beyond standard schema fields.
- Formatted DOCX export with embedded diagram images and browser-based print-to-PDF formatting.
- Iterative document revision via free-text change requests versioned as distinct runs.
- Automated cross-document consistency checking executed prior to finalizing outputs.
- Local file-based versioning, structured run artifact storage, and schema validation with gap-filling.

3.2 Explicitly deferred

- Multi-user accounts and authentication (Current single-user local tool has no need for access control yet).
- Direct synchronization with Jira and Confluence APIs (Initial MVP focuses on self-contained local document generation and export before introducing external SaaS integrations).

Note: These deferred capabilities are candidates for subsequent implementation phases; refer to Section 9 (Future Scope) for details.

4. Detailed Requirements

4.1 Requirements Intake & Authoring Controls

ID	Requirement	Priority
BR-01	User can fill in a structured form describing a project and generate all five deliverables from one shared source of truth	M
BR-02	User can mark any field "auto-generate / enrich" to have the AI fill it in or expand it based on everything else provided	M
BR-03	User can mark a non-critical field "not applicable" to omit its corresponding document section entirely	M
BR-04	User can add custom "Additional Sections" not covered by the standard fields	S

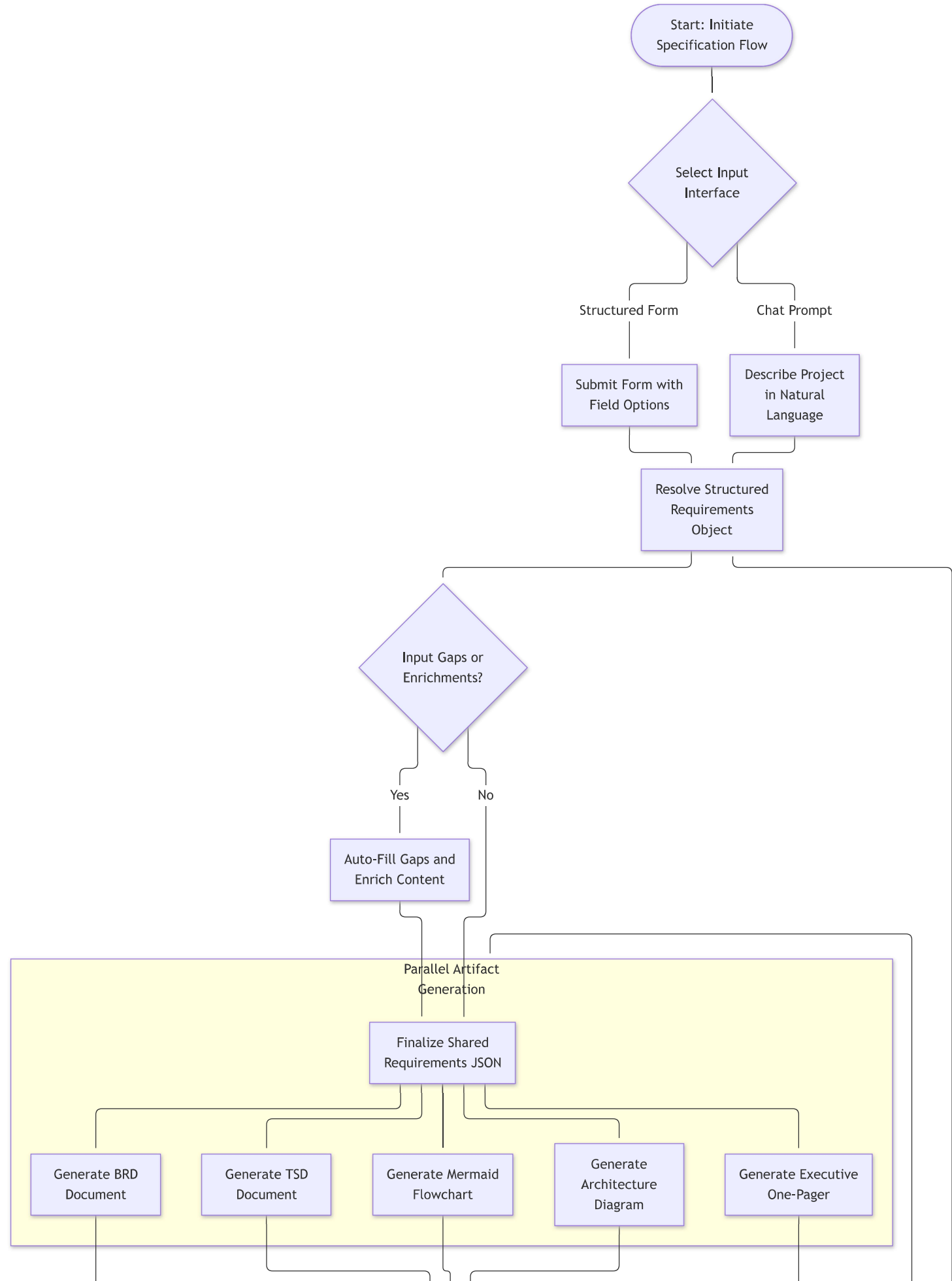
4.2 Document Generation, Validation & Output Lifecycle

ID	Requirement	Priority
BR-05	User can export the BRD, TSD, or one-pager as a DOCX with diagrams embedded as images	M
BR-06	User can request a free-text change after generation, producing a new revision without losing the original	M
BR-07	System shall run an automated consistency check agent across all generated artifacts before presenting final outputs to the user	M
BR-08	User can view side-by-side formatted previews of generated markdown documents and rendered Mermaid diagrams	S

5. Solution Approach

5.1 Process Flow

The project author initiates intake by either entering project details into a structured web form or interacting conversationally with the root chat agent. An extraction, gap-filler, or revision agent processes these inputs to construct a fully validated requirements object. Five specialized generator agents then execute concurrently to produce the BRD, TSD, process flow, architecture diagram, and executive one-pager against that single object. Upon completion, a consistency check agent validates all outputs for cross-document contradictions before persisting versioned artifacts to local disk, where the user can preview, revise, or export them.



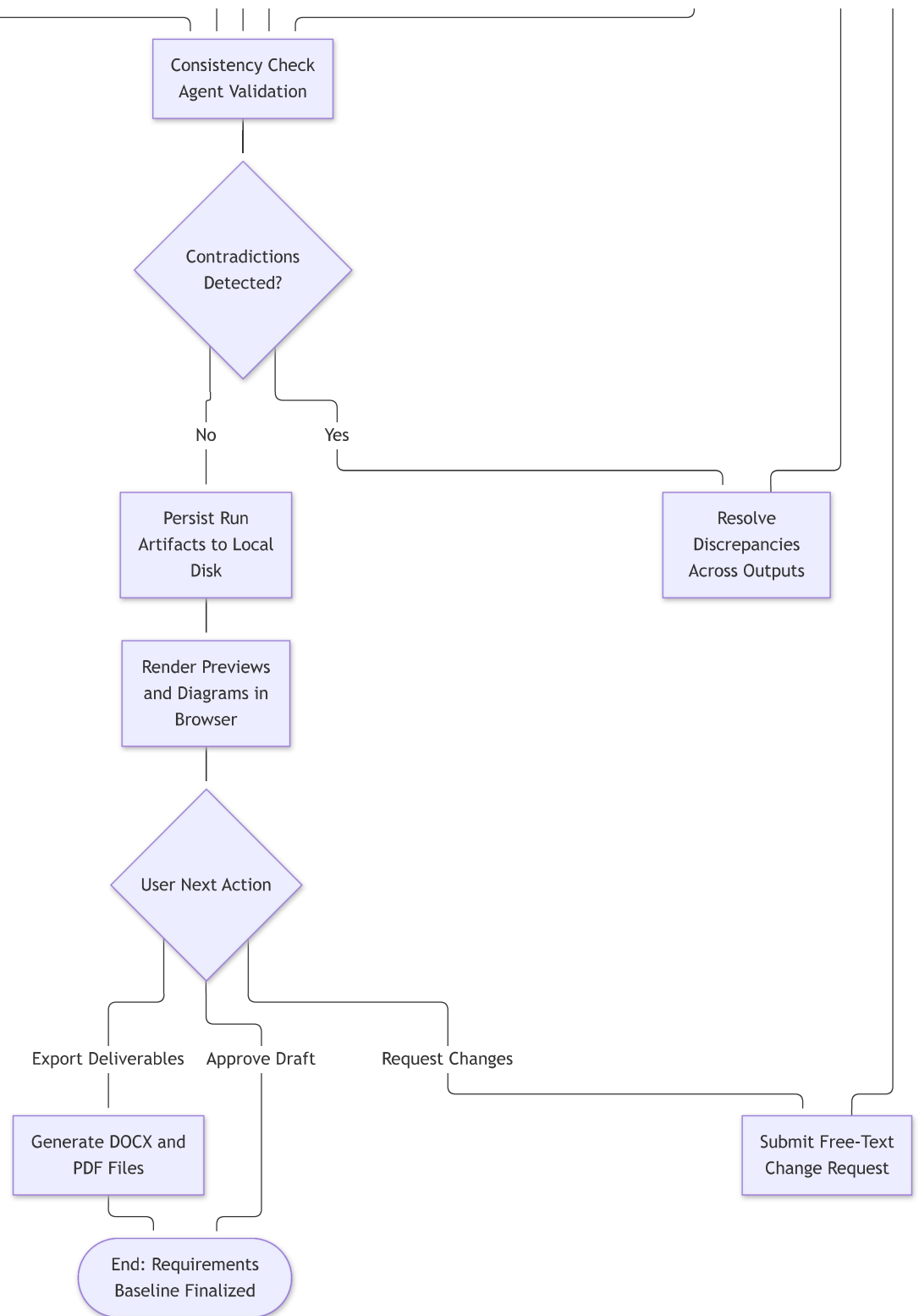
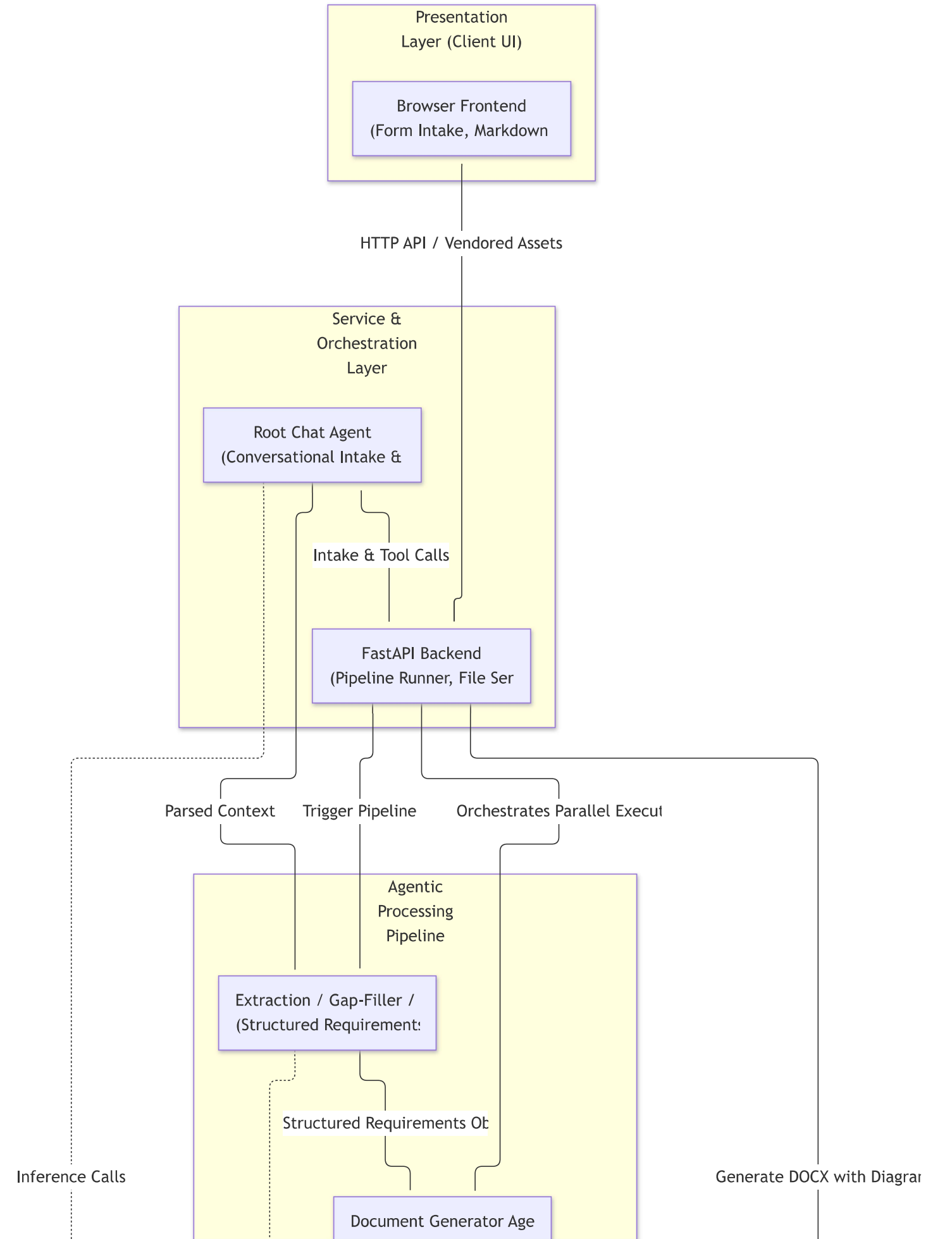


Figure 1 — Process Flow

5.2 Architecture

The solution operates as a locally hosted FastAPI application coordinating a multi-agent orchestration pipeline backed by Google ADK and the Gemini API. The browser frontend presents a vanilla JavaScript interface that renders Markdown and Mermaid diagrams locally without external

dependencies. When intake occurs, the FastAPI backend dispatches tasks through the agent layer—comprising the Root Chat Agent, Extraction/Gap-Filler Agent, Document Generator Agents, and Consistency Check Agent—persisting structured output files to local disk and utilizing pandoc for document compilation.



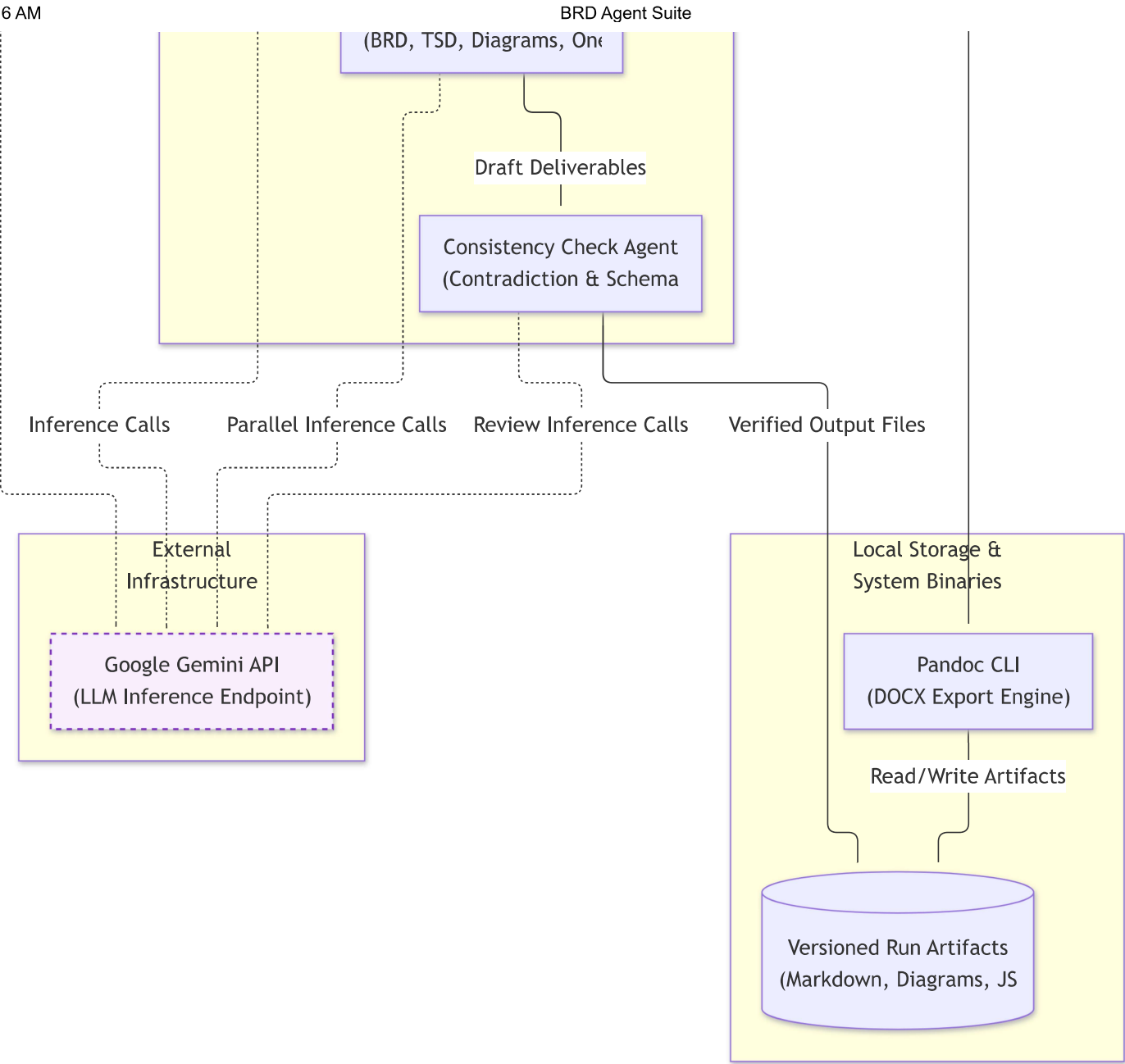


Figure 2 — Architecture

5.3 Proposed Technology Stack

Layer	Proposal	Notes
Core Orchestration & Backend	Python, Google ADK, FastAPI	Provides agent lifecycle management, schema enforcement, and rapid local REST endpoint execution.
User Interface	Vanilla JavaScript, HTML5, CSS3	Eliminates heavy frontend node-module frameworks, ensuring zero-configuration local execution and rapid load times.
Client-Side Rendering	Locally vendored <code>'mermaid.js'</code> and	Enables offline diagram rendering and Markdown previews without outbound CDN or third-party network calls.

Layer	Proposal	Notes
	<code>`marked.js`</code>	
Document Compilation	pandoc CLI	Delivers conversion of Markdown and embedded visual assets into styled DOCX output files.

5.4 Design Principles

- **Single Source of Truth:** All downstream deliverables (BRD, TSD, diagrams, one-pager) must strictly originate from the validated structured requirements object; no generator agent may invent disconnected scope.
- **Air-Gapped Frontend Operation:** The browser interface must execute fully offline once loaded, strictly forbidding runtime CDN requests or third-party telemetry.
- **Isolated Outbound Connectivity:** Network calls are strictly restricted to outbound Gemini API requests using locally managed API credentials.
- **Non-Destructive Evolution:** Iterative changes and regenerations must create new immutable run versions on disk rather than overwriting historical runs.

6. Non-Functional Requirements

6.1 Performance

- Full document generation must complete within 90 seconds for a typical project across all five deliverables.

6.2 Security

- The Gemini API key must be loaded strictly from a local ``.env`` file and never exposed to or processed by the browser frontend.

6.3 Reliability

- LLM generation failures on individual agents must be captured gracefully with retry mechanisms and actionable error reporting presented to the user.

6.4 Usability

- The frontend interface must operate fully offline once loaded, utilizing locally vendored scripts for all Markdown and diagram rendering tasks.

7. Risks

ID	Risk	Impact	Mitigation
R-01	LLM-generated content contains inaccuracies if used without review	High	Every document is clearly marked as a draft requiring explicit human sign-off before operational use.
R-02	The underlying Gemini model is deprecated or renamed	Medium	Model identifiers are centralized in a single configuration constant for instant updates.
R-03	Parallel document generation produces subtle contradictions between documents	Medium	An automated consistency-check agent evaluates all five outputs collectively prior to presenting final results.
R-04	Local host environment lacks required third-party binaries (pandoc or node)	Medium	FastAPI backend validates external CLI prerequisites during startup and presents immediate installation guidance.

8. Delivery Phases

Phase	Content	Exit Criteria
Phase 0 — Core Pipeline	Requirements schema, extraction agent, five document generators, consistency check	All five documents generate correctly from a sample prompt
Phase 1 — Chat Interface	Conversational root agent with clarifying-question policy, served via adk web	Vague prompts trigger batched questions; detailed prompts generate directly
Phase 2 — Structured Form	Form-based generation, diagram rendering, DOCX/PDF export, revision flow	A full generate-view-export-revise cycle works end to end in a browser

Phase	Content	Exit Criteria
Webapp		
Phase 3 — Field-Level Controls	Per-field auto-generate/enrich, mark-not-applicable, custom additional sections	Excluded sections are omitted and renumbered correctly; enrichment adds new content, not just polish

Delivery phases are strictly sequential: Phase 1 depends on the Phase 0 pipeline, Phase 2 builds the web application layer on top of Phase 1 capabilities, and Phase 3 adds granular field controls to the Phase 2 interface.

9. Future Scope (Deferred, Not Rejected)

Nothing in this section is currently committed, estimated, or approved for implementation in the initial release.

Item	What it would unlock	What it would require
Multi-user accounts and authentication	Role-based access control, multi-author collaboration, and centralized enterprise audit trails.	User identity provider integration (OAuth/SAML), database session management, and access-control middleware.
Direct synchronization with Jira and Confluence APIs	Direct export of generated user stories to Jira backlogs and automated publishing of technical specs to Confluence spaces.	Atlassian REST API integration, credential vaulting, schema mapping engines, and network egress permissions.

10. Sign-Off

Role	Name	Signature	Date
Executive Sponsor			
Lead Business Analyst / Product Manager			

Role	Name	Signature	Date
Engineering Lead			

